Project Untitled - API & Feature Documentation

A platform for creating, configuring, and orchestrating AI coding agents.

Table of Contents

1. Agent Types
2. Builder AI
3. API Reference
4. Agent Chaining
5. Webhooks
6. GitHub Integration
7. AI Providers
8. Agent Configuration
9. Playground

Agent Types

Code Agents

Code agents are designed to work with GitHub repositories. They can:

- Clone and work within a repository
- Read, create, and modify files
- Make commits and push changes
- Run shell commands in the sandbox
- Access the full repository context

Best for: Bug fixes, feature development, code refactoring, documentation updates.

Task Agents

Task agents are standalone agents that run prompts without a repository context. They can:

- Execute arbitrary tasks based on system prompts
- Access the web for information retrieval
- Process data and return results
- Be triggered via API for automation

Best for: Data analysis, research tasks, content generation, API integrations, scheduled jobs.

Builder AI

The Builder is a conversational AI assistant that helps you create and configure agents without writing code.

Capabilities

- **Create Agents**: "Make me a task agent that checks stock prices"
- **Configure Agents**: "Enable API access for my agent"
- **List Resources**: "Show me my agents" or "List my repositories"
- **Remember Context**: Stores user preferences for future conversations

Example Conversation

```
User: Make me a code agent for my react-app repo that specializes in testing
Builder: I'll create that for you...
        ✓ Created agent "React App Tester" successfully!

User: Enable API access for it
Builder: Done! API access is now enabled. Here's how to use it...

User: Chain it to my deployment agent
Builder: I've configured the agent to trigger your Deployment Agent on success.
```

API Reference

All API requests require an API key. Generate keys in **Settings > API Keys**.

Authentication

```
Authorization: Bearer YOUR_API_KEY
```

Base URL

```
https://project-untitled.fly.dev/api/v1
```

List Agents

```
GET /api/v1/agents
```

Response:

```
{
  "agents": [
    {
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "name": "Stock Price Checker",
      "agent_type": "task",
      "status": "pending",
      "provider": "claude-code",
      "model": null,
      "api_enabled": true,
      "created_at": "2026-01-10T10:00:00.000Z"
    }
  ]
}
```

Create Agent

```
POST /api/v1/agents
Content-Type: application/json

{
  "name": "My Agent",
  "agent_type": "task",
  "agent_provider": "claude-code",
  "system_prompt": "You are a helpful assistant.",
  "api_enabled": true
}
```

Request Body:

Field	Type	Required	Default	Description
`name`	string	Yes	-	Agent display name
`agent_type`	string	No	`"task"`	`"code"` or `"task"`
`agent_provider`	string	No	`"claude-code"`	`"claude-code"`, `"aider"`, or `"opencode"`
`agent_model`	string	No	null	Model ID (for opencode)
`system_prompt`	string	No	null	Custom instructions
`api_enabled`	boolean	No	`true`	Allow API access
`repo_url`	string	Code agents	-	GitHub repository URL
`repo_name`	string	No	name	Display name
`branch`	string	No	`"main"`	Git branch

Response (201 Created):

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "name": "My Agent",
  "agent_type": "task",
  "status": "pending",
  "provider": "claude-code",
  "model": null,
  "api_enabled": true,
  "created_at": "2026-01-13T10:00:00.000Z"
}
```

Get Agent

```
GET /api/v1/agents/:agentId
```

Note: Returns 403 if `api_enabled` is false for this agent.

Response:

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "name": "my-repo",
  "status": "active",
  "provider": "claude-code",
  "model": null,
  "repo_url": "https://github.com/user/my-repo.git",
  "branch": "main"
}
```

Update Agent

```
PATCH /api/v1/agents/:agentId
Content-Type: application/json

{
  "name": "Updated Name",
  "system_prompt": "New instructions",
  "api_enabled": true,
  "webhook_url": "https://hooks.example.com/notify",
  "chain_to_agent_id": "other-agent-id",
  "chain_condition": "on_success"
}
```

Request Body (all fields optional):

Field	Type	Description
`name`	string	Agent display name
`system_prompt`	string	Custom instructions
`api_enabled`	boolean	Allow API access
`allowed_tools`	array	List of allowed tool names
`webhook_url`	string	URL to call on task completion
`chain_to_agent_id`	string	Agent to trigger next
`chain_condition`	string	`"on_success"`, `"on_failure"`, or `"always"`

Response:

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "name": "Updated Name",
  "system_prompt": "New instructions",
  "api_enabled": true,
  "webhook_url": "https://hooks.example.com/notify",
  "chain_to_agent_id": "other-agent-id",
```

```
  "chain_condition": "on_success"
}
```

Delete Agent

```
DELETE /api/v1/agents/:agentId
```

Response:

```
{
  "success": true,
  "deleted_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Submit Task

```
POST /api/v1/agents/:agentId/tasks
Content-Type: application/json

{
  "prompt": "Check the current price of AAPL stock",
  "priority": 5
}
```

Request Body:

Field	Type	Required	Description
`prompt`	string	Yes	The task instructions
`priority`	number	No	Higher = processed first (default: 0)

Response (202 Accepted):

```
{
  "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "status": "pending",
  "agent_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "created_at": "2026-01-13T10:00:00.000Z"
}
```

Get Task Status

```
GET /api/v1/agents/:agentId/tasks/:taskId
```

Response (pending):

```
{
  "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "status": "pending",
  "result": null,
  "error": null,
  "created_at": "2026-01-13T10:00:00.000Z",
  "started_at": null,
  "completed_at": null
}
```

Response (processing):

```
{
  "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "status": "processing",
  "result": null,
  "error": null,
  "created_at": "2026-01-13T10:00:00.000Z",
  "started_at": "2026-01-13T10:00:05.000Z",
  "completed_at": null
}
```

Response (completed):

```
{
  "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "status": "completed",
  "result": "AAPL (Apple Inc.) is currently trading at $185.42, up 1.2% today.",
  "error": null,
  "created_at": "2026-01-13T10:00:00.000Z",
  "started_at": "2026-01-13T10:00:05.000Z",
  "completed_at": "2026-01-13T10:00:15.000Z"
}
```

Response (failed):

```
{
  "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "status": "failed",
  "result": null,
  "error": "Unable to fetch stock data: API timeout",
  "created_at": "2026-01-13T10:00:00.000Z",
  "started_at": "2026-01-13T10:00:05.000Z",
  "completed_at": "2026-01-13T10:00:20.000Z"
}
```

Task States:

Status	Description

`pending`	Task queued, waiting to be processed
`processing`	Agent is actively working on the task
`completed`	Task finished successfully
`failed`	Task encountered an error

List Tasks

```
GET /api/v1/agents/:agentId/tasks
```

Returns the 50 most recent tasks for the agent.

Response:

```
{
  "tasks": [
    {
      "id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
      "status": "completed",
      "created_at": "2026-01-13T10:00:00.000Z",
      "started_at": "2026-01-13T10:00:05.000Z",
      "completed_at": "2026-01-13T10:00:15.000Z"
    }
  ]
}
```

Complete Example: Submit and Poll

Python

```python
import requests
import time

API_KEY = "your-api-key"
BASE_URL = "https://project-untitled.fly.dev/api/v1"
AGENT_ID = "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

# 1. Submit a task
response = requests.post(
    f"{BASE_URL}/agents/{AGENT_ID}/tasks",
    headers=headers,
    json={"prompt": "Check the current price of AAPL stock"}
```

```python
)
data = response.json()
task_id = data["id"]
print(f"Task submitted: {task_id}")

# 2. Poll for completion
while True:
    response = requests.get(
        f"{BASE_URL}/agents/{AGENT_ID}/tasks/{task_id}",
        headers=headers
    )
    data = response.json()
    status = data["status"]

    print(f"Status: {status}")

    if status == "completed":
        print(f"Result: {data['result']}")
        break
    elif status == "failed":
        print(f"Error: {data['error']}")
        break

    time.sleep(2)
```

Output:

```
Task submitted: t1a2s3k4-i5d6-7890-abcd-ef1234567890
Status: pending
Status: processing
Status: processing
Status: completed
Result: AAPL (Apple Inc.) is currently trading at $185.42, up 1.2% today.
```

JavaScript

```javascript
const API_KEY = "your-api-key";
const BASE_URL = "https://project-untitled.fly.dev/api/v1";
const AGENT_ID = "a1b2c3d4-e5f6-7890-abcd-ef1234567890";

async function runTask(prompt) {
  // Submit task
  const submitRes = await fetch(`${BASE_URL}/agents/${AGENT_ID}/tasks`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${API_KEY}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ prompt })
  });
```

```
  const data = await submitRes.json();
  console.log(`Task submitted: ${data.id}`);

  // Poll for result
  while (true) {
    const statusRes = await fetch(
      `${BASE_URL}/agents/${AGENT_ID}/tasks/${data.id}`,
      { headers: { "Authorization": `Bearer ${API_KEY}` } }
    );

    const task = await statusRes.json();
    console.log(`Status: ${task.status}`);

    if (task.status === "completed") {
      return task.result;
    } else if (task.status === "failed") {
      throw new Error(task.error);
    }

    await new Promise(r => setTimeout(r, 2000));
  }
}

// Usage
runTask("Check the current price of AAPL stock")
  .then(result => console.log("Result:", result))
  .catch(err => console.error("Error:", err));
```

cURL

```
# Submit a task
curl -X POST "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID/tasks" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Check AAPL stock price"}'

# Response: {"id":"task-id","status":"pending","agent_id":"...","created_at":"..."}

# Poll for status
curl "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID/tasks/TASK_ID" \
  -H "Authorization: Bearer YOUR_API_KEY"

# Response: {"id":"...","status":"completed","result":"AAPL is trading at
$185.42",...}
```

Agent Chaining

Chain agents together to create multi-step workflows. When one agent completes, it automatically triggers the next.

Configuration

Use the Update Agent endpoint or the Configure tab in the UI:

```
curl -X PATCH "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "chain_to_agent_id": "next-agent-id",
    "chain_condition": "on_success"
  }'
```

Chain Conditions

Condition	Description
`on_success`	Trigger next agent only if task succeeds
`on_failure`	Trigger next agent only if task fails
`always`	Trigger next agent regardless of outcome

How It Works

1. Agent A completes a task
2. System checks for chain configuration
3. If conditions are met, creates a new task for Agent B
4. Agent B receives context from Agent A's output

Chain Prompt Format

The chained agent receives:

On Success:

```
[Chained from previous agent]

Previous task: <original prompt>

Previous result:
<Agent A's output>

Continue the work based on the above context.
```

On Failure:

```
[Chained from previous agent - FAILED]

Previous task: <original prompt>

Error:
<error message>
```

```
Handle or recover from the above error.
```

Example: Data Pipeline

```
Data Collector Agent (chain_condition: on_success)
    ↓
Data Analyzer Agent (chain_condition: on_success)
    ↓
Report Generator Agent
```

Webhooks

Get notified when agent tasks complete instead of polling.

Configuration

Set a webhook URL via the API or in the agent's Configure tab:

```
curl -X PATCH "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"webhook_url": "https://hooks.example.com/notify"}'
```

Webhook Payload

Your endpoint will receive a POST request when a task **completes successfully**:

```
{
  "task_id": "t1a2s3k4-i5d6-7890-abcd-ef1234567890",
  "agent_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "status": "completed",
  "result": "AAPL is currently trading at $185.42"
}
```

Note: Webhooks are currently only triggered on successful task completion. For failed tasks, use polling or agent chaining with `on_failure` condition.

Use Cases

- Slack/Discord notifications
- Trigger external workflows (Zapier, n8n, etc.)
- Update dashboards
- Log to monitoring systems

GitHub Integration

Connect your GitHub account to enable code agents.

Setup

1. Go to **Settings**
2. Click **Connect GitHub**
3. Authorize the application
4. Your repositories will be available for code agents

Capabilities

- **Repository Access**: Clone any connected repo
- **Branch Selection**: Work on specific branches
- **Commits**: Agents can commit changes
- **Push**: Push changes back to GitHub

AI Providers

Choose from multiple AI providers for your agents.

Claude Code (Default)

Anthropic's agentic coding CLI. Most capable for complex coding tasks.

- **Best for**: Complex refactoring, multi-file changes, architectural decisions

Aider

AI pair programming powered by OpenAI.

- **Best for**: Interactive coding sessions, quick fixes

OpenCode

Supports 75+ providers and 100k+ models including:

Provider	Example Models
OpenRouter	DeepSeek, Llama, Mistral
Together AI	Llama, Mixtral
DeepSeek	DeepSeek Chat, Coder
Google	Gemini Pro, Flash
Groq	Llama, Mixtral (fast inference)
Mistral	Mistral Large, Medium
xAI	Grok

Agent Configuration

System Prompt

Custom instructions that define the agent's behavior and focus.

Example - Security Reviewer:

```
You are a security-focused code reviewer.
When reviewing code:
- Check for SQL injection vulnerabilities
- Look for XSS attack vectors
- Identify insecure authentication patterns
- Flag hardcoded secrets
```

Example - Stock Checker:

```
You are a financial data assistant.
When asked to check stock prices:
- Fetch current trading price
- Include daily change percentage
- Note market status (open/closed)
- Format numbers with proper currency symbols
```

Allowed Tools

Restrict which tools the agent can use:

Tool	Description
`Read`	Read files
`Write`	Write/modify files
`Edit`	Edit existing files
`Bash`	Run shell commands
`Glob`	Search for files
`Grep`	Search file contents
`LS`	List directories

Secrets

Store sensitive values that get injected into the agent's environment:

```
{
  "DATABASE_URL": "postgres://...",
  "EXTERNAL_API_KEY": "secret-key-here"
}
```

Playground

Interactive testing environment for task agents.

Features

- **Live Output**: Stream agent responses in real-time
- **Configuration Preview**: See system prompt and tools that will be used
- **Task History**: View previous runs and results
- **Auto-Start**: Sandbox starts automatically when you run a task

Default Behavior

- Default prompt is `execute` - runs the agent with its configured system prompt
- Edit the prompt to customize the request
- Cancel button to stop waiting for long-running tasks

Error Responses

All endpoints may return these errors:

Status	Error	Description
400	`"Name is required"`	Missing required field
400	`"Prompt is required"`	Missing prompt for task
400	`"repo_url is required for code agents"`	Code agent needs repo
403	`"API access not enabled for this agent"`	Agent's api_enabled is false
404	`"Agent not found"`	Invalid agent ID or not owned by user
404	`"Task not found"`	Invalid task ID

Quick Start

1. Get Your API Key

Go to **Settings > API Keys** and create a new key.

2. Create an Agent

```
curl -X POST "https://project-untitled.fly.dev/api/v1/agents" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Stock Checker",
    "agent_type": "task",
    "system_prompt": "You check stock prices and provide financial data."
  }'
```

3. Submit a Task

```
curl -X POST "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID/tasks" \
  -H "Authorization: Bearer YOUR_API_KEY" \
```

```
  -H "Content-Type: application/json" \
  -d '{"prompt": "Check AAPL stock price"}'
```

4. Get Results

```
curl "https://project-untitled.fly.dev/api/v1/agents/AGENT_ID/tasks/TASK_ID" \
  -H "Authorization: Bearer YOUR_API_KEY"
```

Last updated: January 2026